

Geometry

lanos212

lanos.
2115

讲课人是如何确定的

不言而喻，一目了然。

前言

计算几何往往在 XCPC, Codeforces 等比赛中出现较多,
而由于众所周知的原因, 在 NOI 系列比赛中出现频率并不高。

其相关题目可能具有以下特点:

- 这不是直接来题, 没啥意思。
- 我精度被卡了??
- 怎么板子题放放。
- 原来是科技题。
- 我也就写了 10KB, 感觉还好。
- 怎么还有三点共线??

一般你把**该学的都学了**, 并且有一定的**代码实现能力**即可。



温馨提示

- 直接读入浮点数类型非常慢， 10^5 级别的数据就可能 TLE，如果数据是整数，请务必读入整数类型然后再赋值。
- 如果有条件的话最好开 `long double`，形如值域 10^9 ，精度 10^{-10} 的二分如果只开 `double` 就爆了。
- 实际使用时凸包上的每个角都是 $< 180^\circ - \epsilon$ ，不是 $< 180^\circ + \epsilon$ 。
- `sqrt`，`sqrtl` 等函数是 $O(\log |V|)$ 的，只是在 `-O2` 下常数小。
- 顺带一提，不要有人再把 `-O2` 打成 `-o2` 了。



如何表示直线

在计算几何题中，我们基本都使用**点和向量**解决问题。

如果用 $Ax + By + C = 0$ 表示直线，或者甚至是 $y = kx + b$ ，会出现以下问题：

- 运算往往需要解方程，比较麻烦，特别是容易出现计算错误。
- 遇到与坐标轴平行的情况容易出现问題。

在使用**点和向量**解决问题时，我们往往用**一个点和一条模长不为 0 的向量**表示一条直线/射线/线段。



从 0 开始写计算几何基础模板

既然我们打算用**点和向量**解决问题，在实现上，我们会对它们分别封装一个类。

```
struct V{
    double x,y;
    V(double nx=0,double ny=0){x=nx,y=ny;}
    friend double operator ^(V A,V B){return A.x*B.x+A.y*B.y;}
    friend double operator *(V A,V B){return A.x*B.y-A.y*B.x;}
    friend V operator +(V A,V B){return V(A.x+B.x,A.y+B.y);}
    inline double len(){return __builtin_sqrtl(x*x+y*y);}
};

struct P{
    double x,y;
    P(double nx=0,double ny=0){x=nx,y=ny;}
    friend V operator -(P A,P B){return V(A.x-B.x,A.y-B.y);}
    friend P operator +(P A,V B){return P(A.x+B.x,A.y+B.y);}
};

inline double dist(P A,P B){return __builtin_sqrtl((A.x-B.x)*(A.x-B.x)+(A.y-B.y)*(A.y-B.y));}
```

从 0 开始写计算几何基础模板

直线可以用一个点和一条模长不为 0 的向量表示。

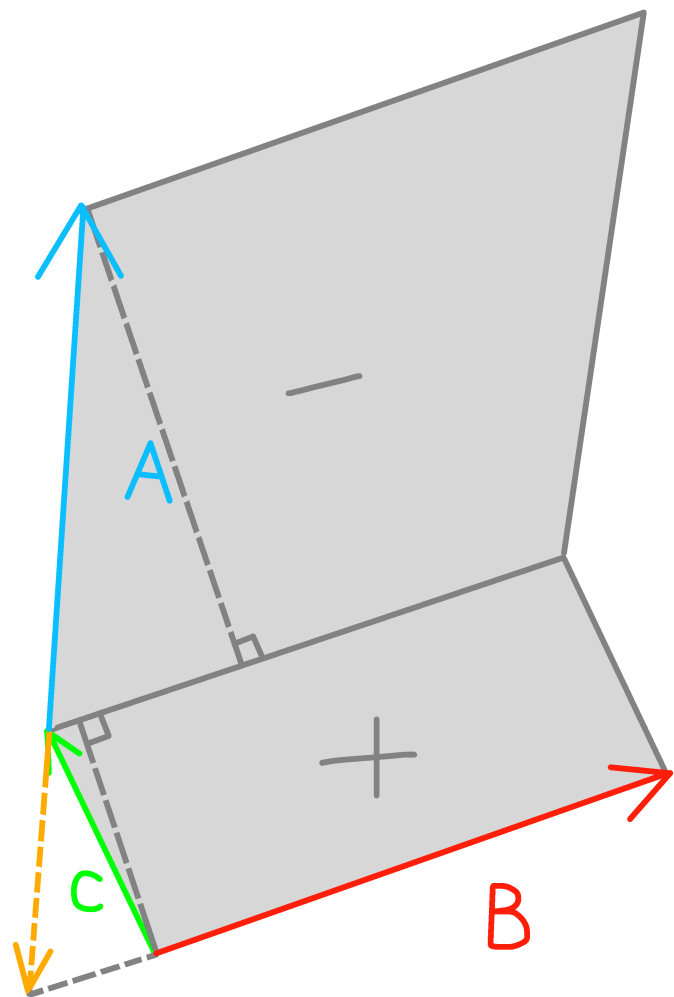
```
struct L{
    P s; V to;
    L(P ns=P(),V nto=V()){s=ns,to=nto;}
    inline double len(){return to.len();}
};
```



从 0 开始写计算几何基础模板

用向量做两直线求交点。

如下图，用两个叉积得到面积之比，即得到高线之比，以此求出连向交点的黄色向量。



Umos.

从 0 开始写计算几何基础模板

用向量做两直线求交点。

```
inline P cross(L A,L B){  
    V C=A.s-B.s;  
    double t=(B.to*C)/(A.to*B.to);  
    return P(A.s.x+t*A.to.x,A.s.y+t*A.to.y);  
}
```



从 0 开始写计算几何基础模板

最后是浮点数的精度问题。

我们往往在比较时，用这样一个函数来判断一个数与 0 的大小关系。

```
inline int ck(double val){  
    if (fabs(val)<1e-12) return 0;  
    return (val>0?1:-1);  
}
```



从 0 开始写计算几何基础模板

还有一个重要的常数！

如果你曾经闲着没事干，那么你应该会背

```
 $\pi=3.1415926535897932384626433832795028841971$   
6939937510582097494459230781640628620899...
```

当然我们实际上不需要这么干，只需要调用 `acos` 函数就可以了。

```
const double PI=acos(-1);`
```

Unos.
= 115

正余弦定理

正弦定理

在三角形 $\triangle ABC$ 中, 若角 A, B, C 所对边分别为 a, b, c , 则有:

$$\frac{a}{\sin A} = \frac{b}{\sin B} = \frac{c}{\sin C} = 2R$$

其中, R 为 $\triangle ABC$ 的外接圆半径。

余弦定理

在三角形 $\triangle ABC$ 中, 若角 A, B, C 所对边分别为 a, b, c , 则有:

$$a^2 = b^2 + c^2 - 2bc \cos A$$

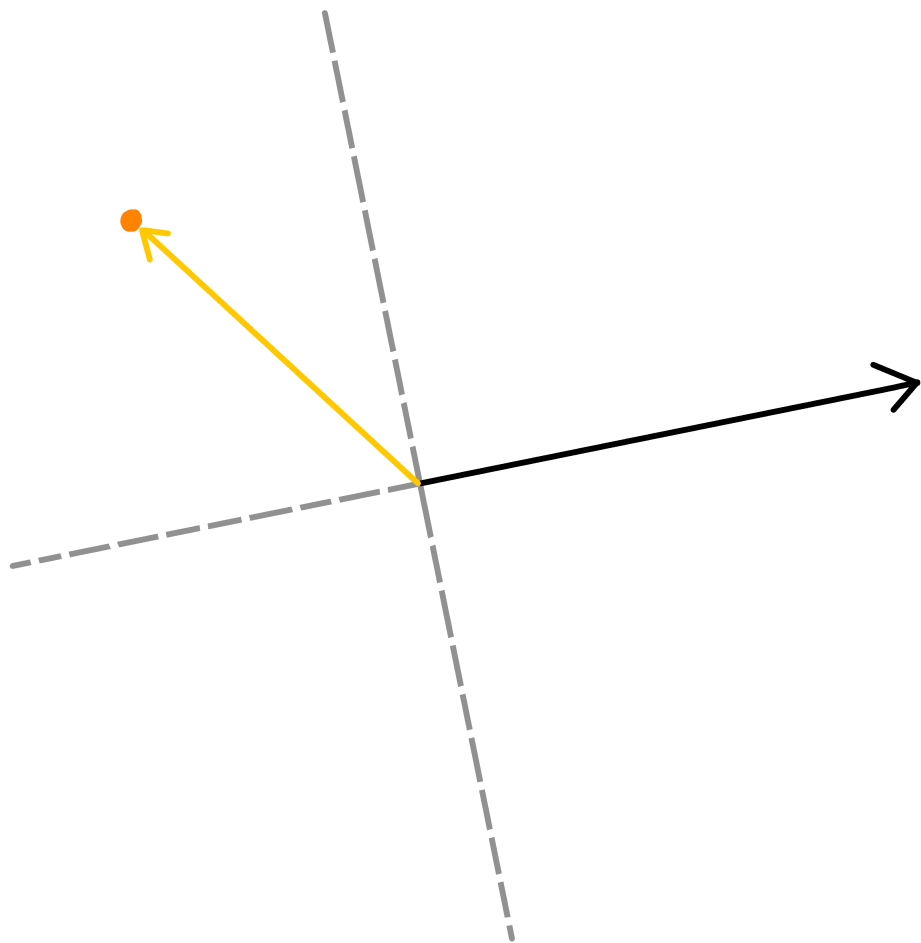
$$b^2 = a^2 + c^2 - 2ac \cos B$$

$$c^2 = a^2 + b^2 - 2ab \cos C$$



射线与点的位置关系

将表示射线的向量与射线的端点连向给定点得到的向量进行点积与叉积，根据结果正负性即可判断点位于如图四个区域中的哪一部分。



旋转一个向量

使用 `atan2` 函数，传入向量的两个参数，

`atan2(y,x)` 可以返回一个在 $[-\pi, +\pi]$ 内的向量与 x 轴形成的夹角大小。

将夹角大小加上你要的旋转角度。

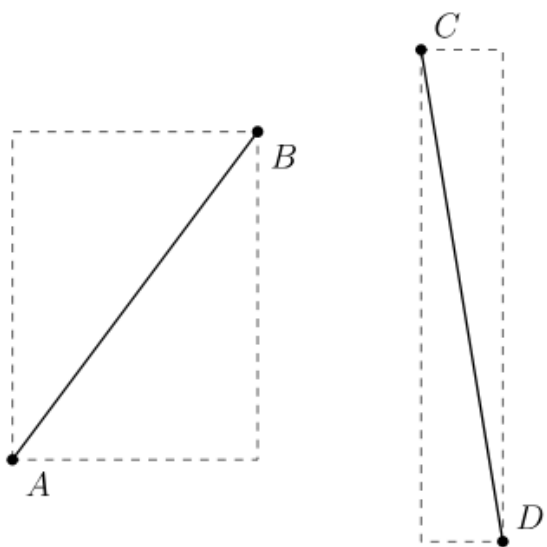
设此时角度为 θ ，向量模长为 L ，那么旋转后的向量就是 $\begin{bmatrix} L \cos \theta \\ L \sin \theta \end{bmatrix}$ 。



快速排斥+跨立实验

满足下面两个实验，是两条线段**不交**的充要条件：

- **快速排斥实验**：两个分别包含给定线段的边平行于坐标轴的最小矩形**不交**。



- **跨立实验**：其中一条线段的两端都在另一条线段所在直线的**同侧**。



判断点是否在多边形内部

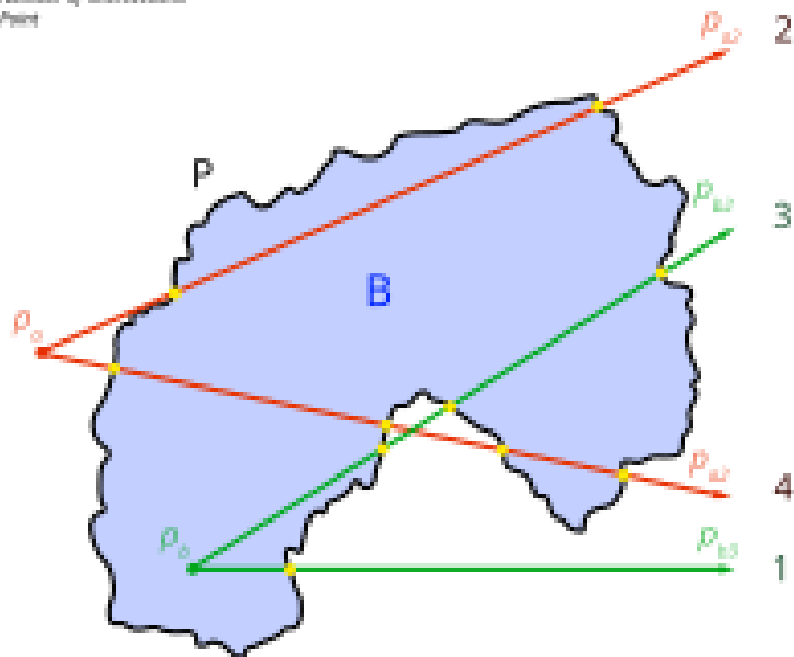
方法 1: 以该点作为端点作一条射线, 看与多边形的边界相交次数。

如果为奇数, 则在内部, 否则在外部。

在边界上的情况请根据实际要求进行特判。

The Jordan Curve Theorem for Polygons

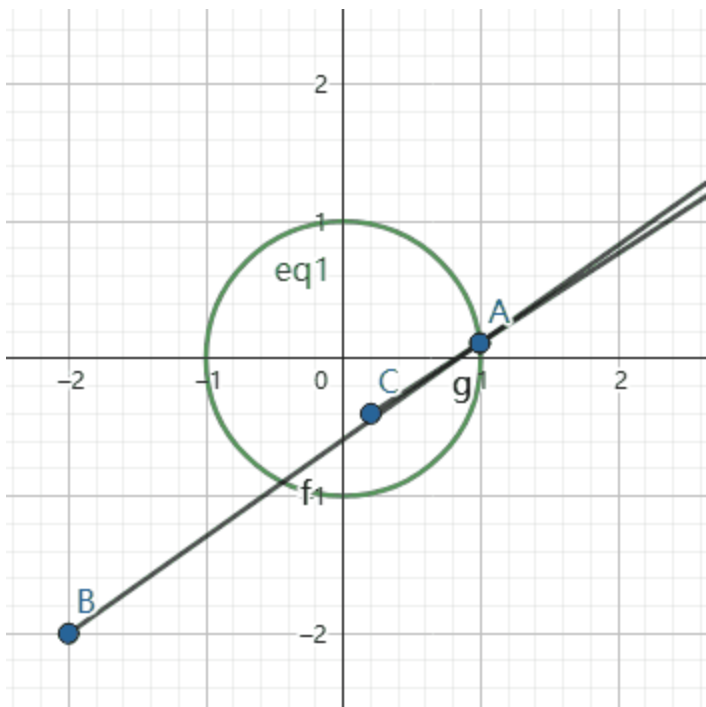
- A - Inside
- P - Polygon
- $\bullet$ - Intersection
- L - Number of Intersections
- P_o - Point



判断点是否在多边形内部

方法 2： 将一个动点按照逆时针方向绕多边形边界一圈，将给定点连向动点，看这条射线的旋转总角度，如果为 360° 则在内部，为 0° 则在外部。

在边界上的情况请根据实际要求进行特判。（若无法渲染，点击图片打开 GIF）



Umos.
三 1 1 5

求多边形面积

从多边形上某个点开始，按照逆时针顺序将若干个三角形面积相加即可。

(若该三角形是顺时针反过来绕的，面积看作是一个负数)

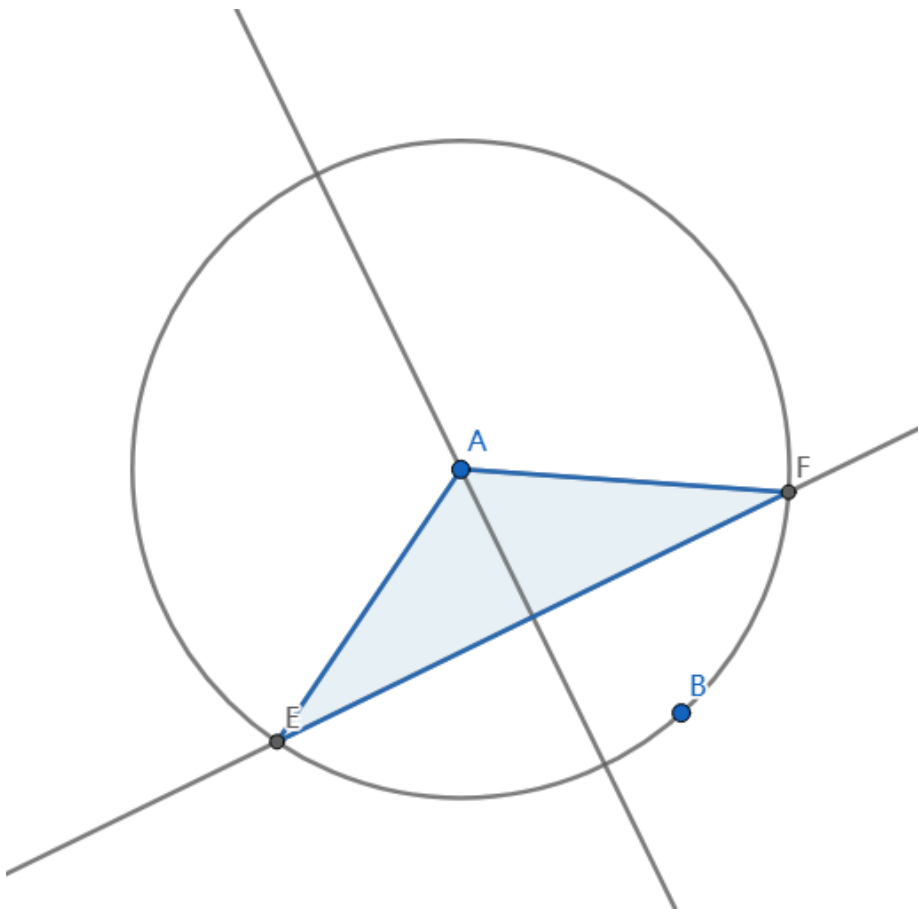
三角形面积直接用 $\frac{\text{叉积的值}}{2}$ 即可，这样把正负性也顺带算上了。

由此也可得到一个结论：**若干整点围成的多边形面积的两倍一定是整数。**



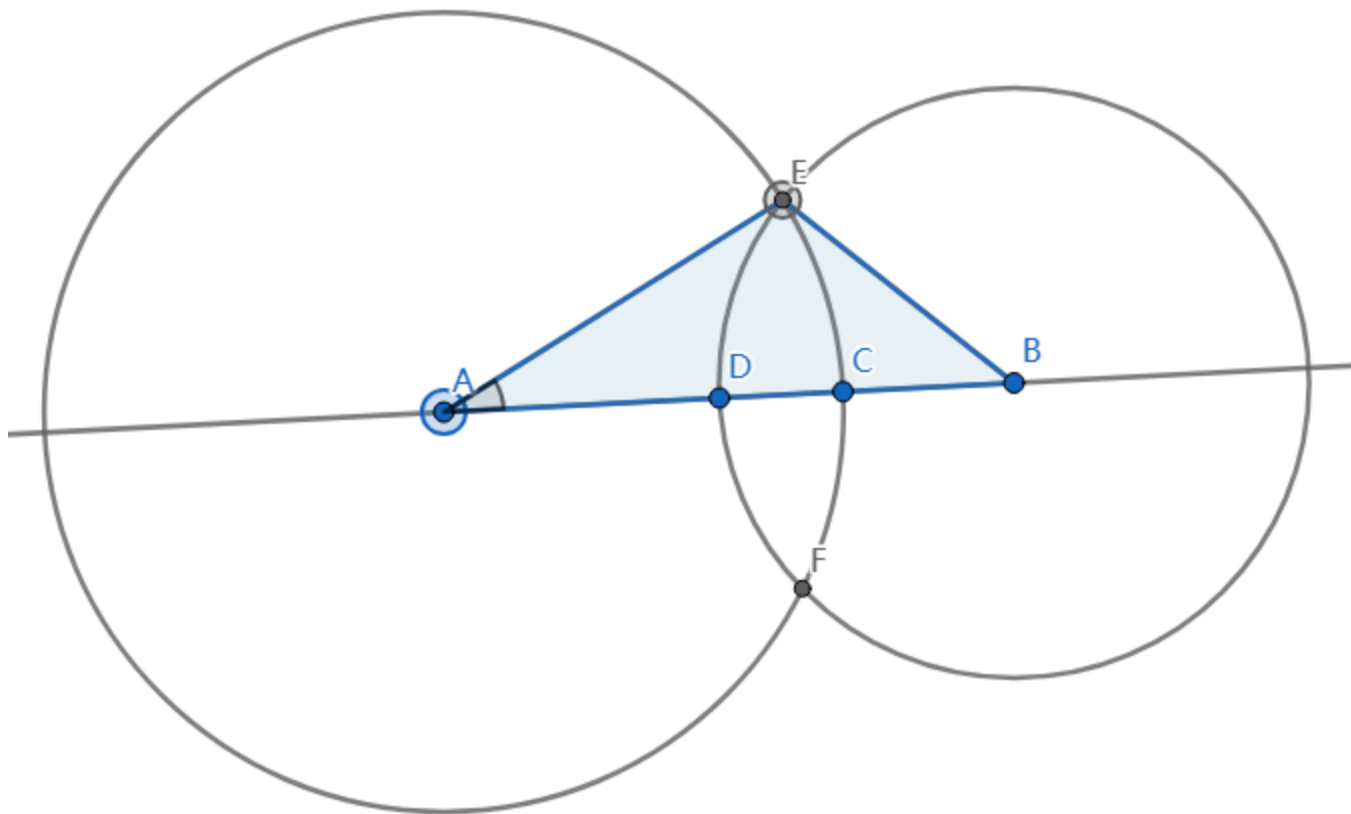
圆与直线交点

- 作出圆心到直线的垂线。
- 比较半径与垂线段的长度，特判不交的情况。
- 其余情况有一或两个交点，勾股出交点与垂足的距离即可，距离为 0 时相切。



两圆交点

- 用一条直线连接两个圆心，根据圆心距特判不交的情况。
- 其余情况有一或两个交点，在如图三角形上使用余弦定理得到 $\angle BAE$ 大小。
- 通过对 $\overrightarrow{AC}$ 往两个方向旋转该角度得到交点，若 $\angle BAE = k\pi$ ($k \in \mathbb{Z}$) 则相切。



距离

在 n 维空间内有两点 $A(a_1, a_2, \dots, a_n), B(b_1, b_2, \dots, b_n)$ 。

欧几里得距离（欧氏距离）：

$$|AB| = \sqrt{\sum_{i=1}^n (a_i - b_i)^2}$$

曼哈顿距离：

$$d(A, B) = \sum_{i=1}^n |a_i - b_i|$$

切比雪夫距离：

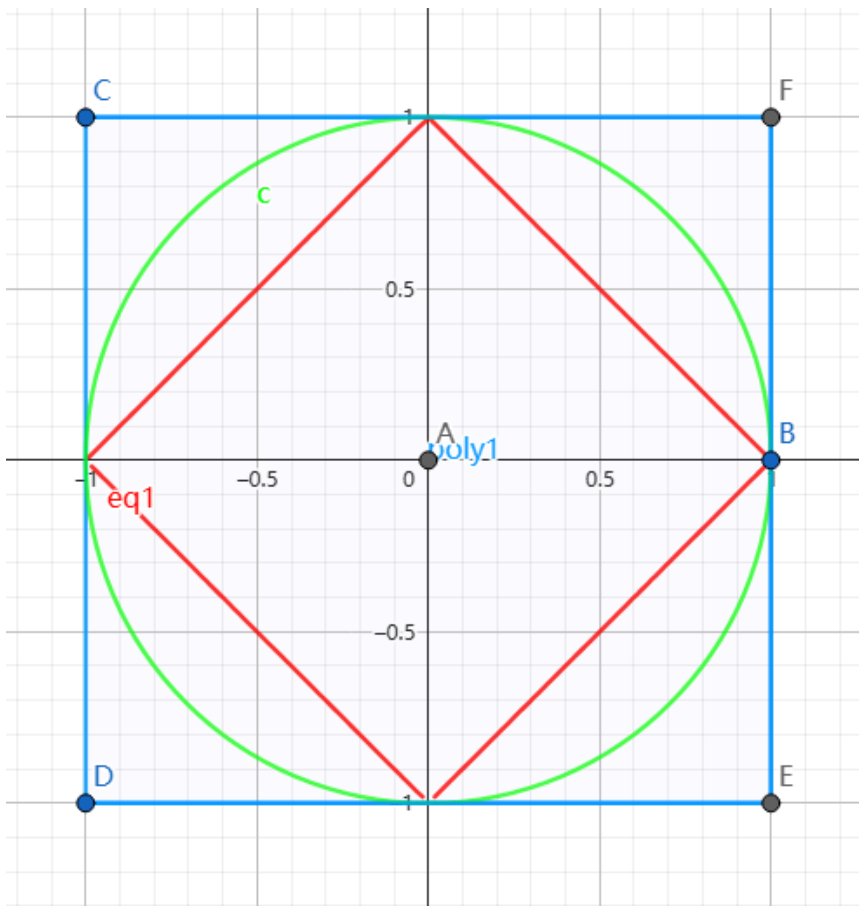
$$d(A, B) = \max_{i=1}^n |a_i - b_i|$$



距离

下图二维空间内为所有距离原点 1 单位长度的点围成的形状。

(欧几里得距离为绿色，曼哈顿距离为红色，切比雪夫距离为蓝色)



距离

不难看出，**曼哈顿距离**和**切比雪夫距离**在将坐标轴旋转 45° 后，进行 $\sqrt{2}$ 比例的放缩，那么它们之间就可以互相转化。

在高维空间内同理，因为围成的空间一直是一个超立方体。

请灵活运用两者的性质，在需要的时候进行转化即可。

转化示例：

- [IOI2007] pairs 动物对数，求曼哈顿距离在定值内的点对数。
转化为切比雪夫距离后，变成高维数点问题。
- [TJOI2013] 松鼠聚会，求一个点，使得其对于给定点集内每个点的切比雪夫距离和最小。转化为曼哈顿距离后，两轴可以独立计算。



某经典 Trick

求 $F(x) = \sum_{i=1}^n |a_i - x|$ 这一函数的极值点以及极值。

结论是将 a 从小到大排序, 那么 $x \in [a_{\frac{n}{2}}, a_{\frac{n}{2}+1}]$ 。

证明考虑放到数轴上, 这个区间内 x 左右点数相同, 而往这个区间外移动, 一定只会让总距离变得更大。



某经典 Trick

[Tenka1-2017 E] CARtesian Coodinate

Time Limit: 5 sec / Memory Limit: 256 MB

给你 n 条直线, 第 i 条解析式为 $A_i x + B_i y = C_i$ 。

保证直线之间两两相交, 且不与坐标轴平行。

求一个点, 使得其到 $\frac{n(n-1)}{2}$ 个直线间交点的总曼哈顿距离最小。

如果有多个点, 请输出横纵坐标之和最小的一个。

数据范围

- $2 \leq n \leq 4 \times 10^4$
- $1 \leq |A_i|, |B_i| \leq 10^4$
- $0 \leq |C_i| \leq 10^4$
- 读入的数都为整数。



某经典 Trick

设 $m = \frac{n(n-1)}{2}$, $k = \lfloor \frac{m+1}{2} \rfloor$ 。

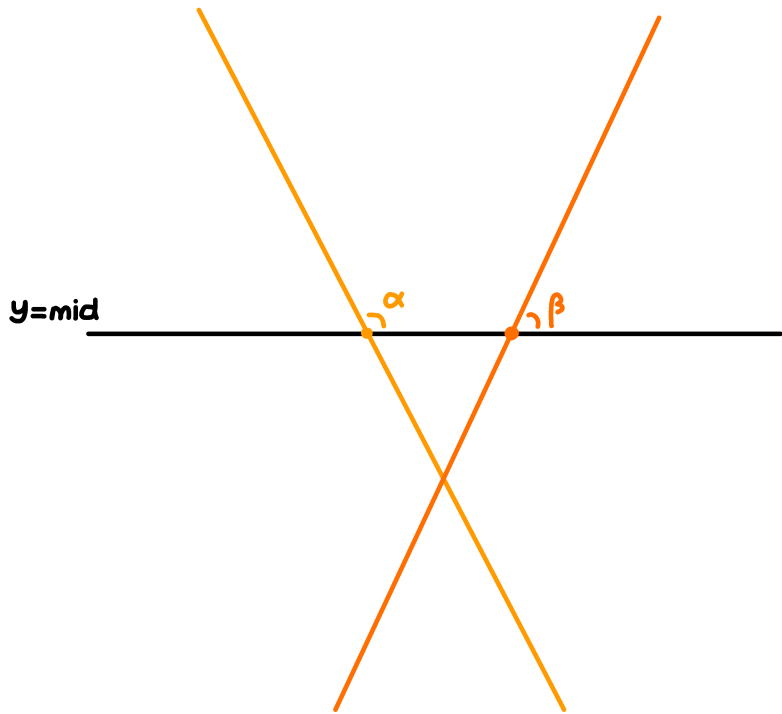
我们现在要求所有 m 个交点中，横/纵坐标的第 k 小值。

这里只说明纵坐标第 k 小值的求法，横坐标只要交换 A_i, B_i 即可。

考虑二分答案，现在要判断交点在 $y = mid$ 下方的个数是否至少有 k 个。



某经典 Trick



将直线按照与 $y = mid$ 交点的横坐标排序，那么两直线相交的充要条件是，它们与 $y = mid$ 在右上方的夹角大小形成**逆序对**。

离散化后使用树状数组数逆序对，时间复杂度 $O(n \log n (\log |V| + |\log \epsilon|))$ 。

UNOS.
= 115

Pick 定理

给定**顶点均为整点**的简单多边形，皮克定理说明了其面积 A 和内部格点数目 i ，边上格点数目 b 的关系：

$$A = i + \frac{b}{2} - 1$$

大致证明思路：（下面图形都默认顶点都为整点）

- 证明直角三角形和长方形下定理正确。
- 证明将一个图形割下一块，若割下部分和原图形满足定理，则剩余图形也满足。
- 由于上面两条，得到任意三角形都满足定理。
- 证明将一个图形割成两部分，若两部分都满足定理，则原图形也满足。
- 将简单多边形三角剖分，由前两条得到任意多边形都满足定理。

具体每步证明时的计算是平凡的，这里不赘述。



二维凸包

凸多边形是指所有内角大小都在 $[0, \pi]$ 范围内的**简单多边形**。

在平面上能包含给定点集的最小凸多边形叫做**凸包**。

凸包是包含给定点集的所有图形中，**周长最小**的。

(你可以想象是用橡皮筋包住了这些点)

实际求凸包时，我们会让每个角都 $< \pi$ ，因为带三点共线的凸包在实际使用时往往会出很多细节问题。



二维凸包

Andrew 算法可以在 $O(n \log n)$ 的时间复杂度内求 n 个平面上点的凸包。

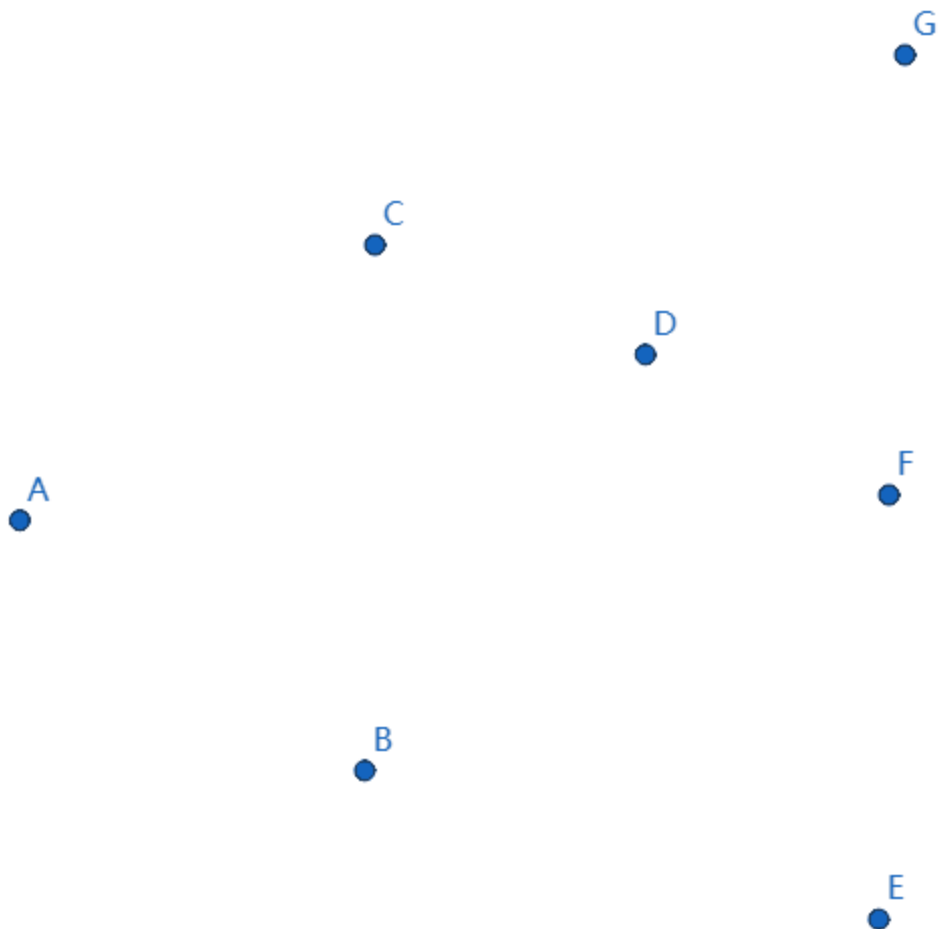
其流程为：

- 将所有点以横坐标为第一关键字，纵坐标为第二关键字从小到大排序。此时第一个和最后一个点一定在凸包内。
- 从左往右用单调栈求这些点的下凸壳。
- 从右往左用单调栈求这些点的上凸壳。
- 两个凸壳拼起来就是你要求的凸包。

时间复杂度瓶颈在于排序，常数较小。



二维凸包



(若无法渲染，点击图片打开 GIF)

二维凸包

Graham **扫描法**做法类似，时间复杂度也为 $O(n \log n)$ 。

其流程为：

- 取一个纵坐标最小的点，这个点一定在凸包内。
- 以该点为基准，剩余点将极角大小作为第一关键字，与基准点的距离作为第二关键字从小到大排序。
- 用单调栈扫一遍即可。

时间复杂度瓶颈在于排序。



如何快速判断某点是否在凸包内

用一条直线把凸包划分成两个凸壳。

判断点在哪一侧之后，用你喜欢的方式去二分即可，单次查询 $O(\log n)$ 。



平面最近点对

这是一个经典计算几何问题，要求在平面上 n 个点中选出 2 个，使得它们的距离最小。

有**经典分治做法**，时间复杂度 $O(n \log n)$ 。

以及一个**期望线性做法**，期望时间复杂度 $O(n)$ ，但是常数很大，在 OI 的常见数据范围下效率往往不如前者。



平面最近点对 - 经典分治做法

先将所有点按照横坐标从小到大排序，考虑进行 CDQ 分治。

假设当前正在求解区间 $[l, r]$ 内的最近点对。

流程如下：

- 令 $mid = \lfloor \frac{l+r}{2} \rfloor$ ，先求解 $[l, mid]$ 与 $[mid + 1, r]$ 内的最近点对。
- 设此时两侧内最近点对距离为 D ，现在考虑左侧与右侧之间的最近点对。
- 将两侧点按照纵坐标从小到大排序，设左侧横坐标最大为 lim ，称右侧某个点为**关键点**，当且仅当这个点的横坐标 $< lim + D$ 。
- 对于左侧某个纵坐标为 y 的点，只找右侧**关键点**中，纵坐标在 $(y - D, y + D)$ 区间内的点更新答案。



平面最近点对 - 经典分治做法

为了保证时间复杂度的实现方式：

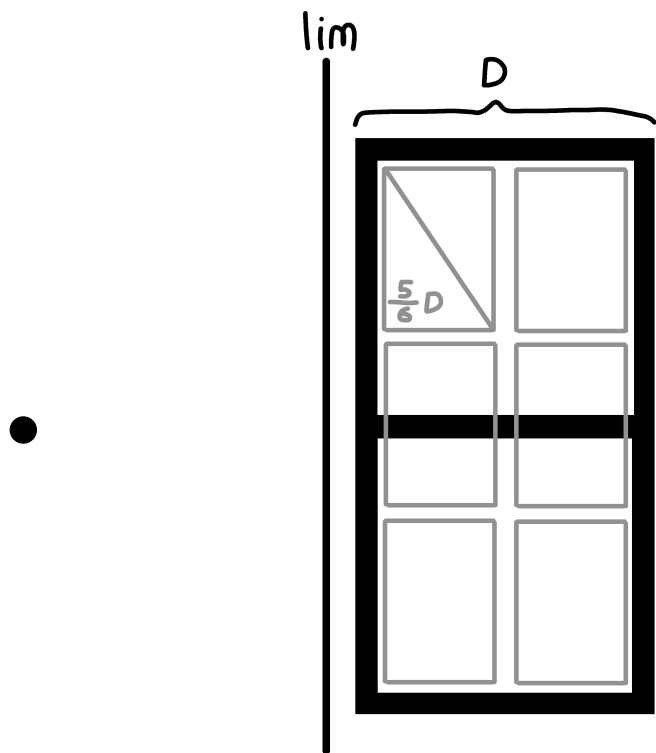
- 由于两边已按纵坐标排序，对于左侧点找右侧区间的时间复杂度是线性的。
- 可以证明，对于每个左侧点，只会找右侧至多 6 个关键点。（应该可以证到 5? ）
- 区间求解完之后，左右两侧的点直接按照纵坐标归并排序，时间复杂度线性。
- 根据 $T(n) = 2T(\frac{n}{2}) + O(n)$ ，分析得到时间复杂度为 $T(n) = O(n \log n)$ 。



平面最近点对 - 经典分治做法

对于每个左侧点，只会找右侧至多 6 个关键点的证明：

- 如图，左侧某个点只会找到右侧图示黑色区域内的点。
- 将右侧黑色区域划分为图示的 6 个灰色部分，每一部分的对角线为 $\frac{5}{6}D$ 。
- 故每一灰色部分至多只有 1 个点，黑色区域内总共至多有 6 个点。



平面最近点对 - 期望线性做法

该做法流程如下：

- 先把所有点前后顺序打乱。
- 考虑将一个个点按顺序加进去，设加入第 i 个点时，之前点的最近点对距离为 D 。
- 将平面划分成无限个 $D \times D$ 的网格。
- 在第 i 个点所在网格的周围 3×3 个网格找点更新答案。（用哈希表存网格与点）
- 一个网格内至多 3 个点，这样至多会找 27 个点。
- 如果此时更新了答案，那么就用 $O(i)$ 的时间复杂度重构网格。
- i 出现在前 i 个点的最近公共点对的概率为 $\frac{2}{i}$ ，
从而第 i 个点期望时间代价为 $O(1)$ ，故总期望时间复杂度 $O(n)$ 。



旋转卡壳

我们直接用模板题引入这一算法。

Time Limit: 1 sec / Memory Limit: 512 MB

给出平面上 n 个点，求凸包直径。（平面最远点对）

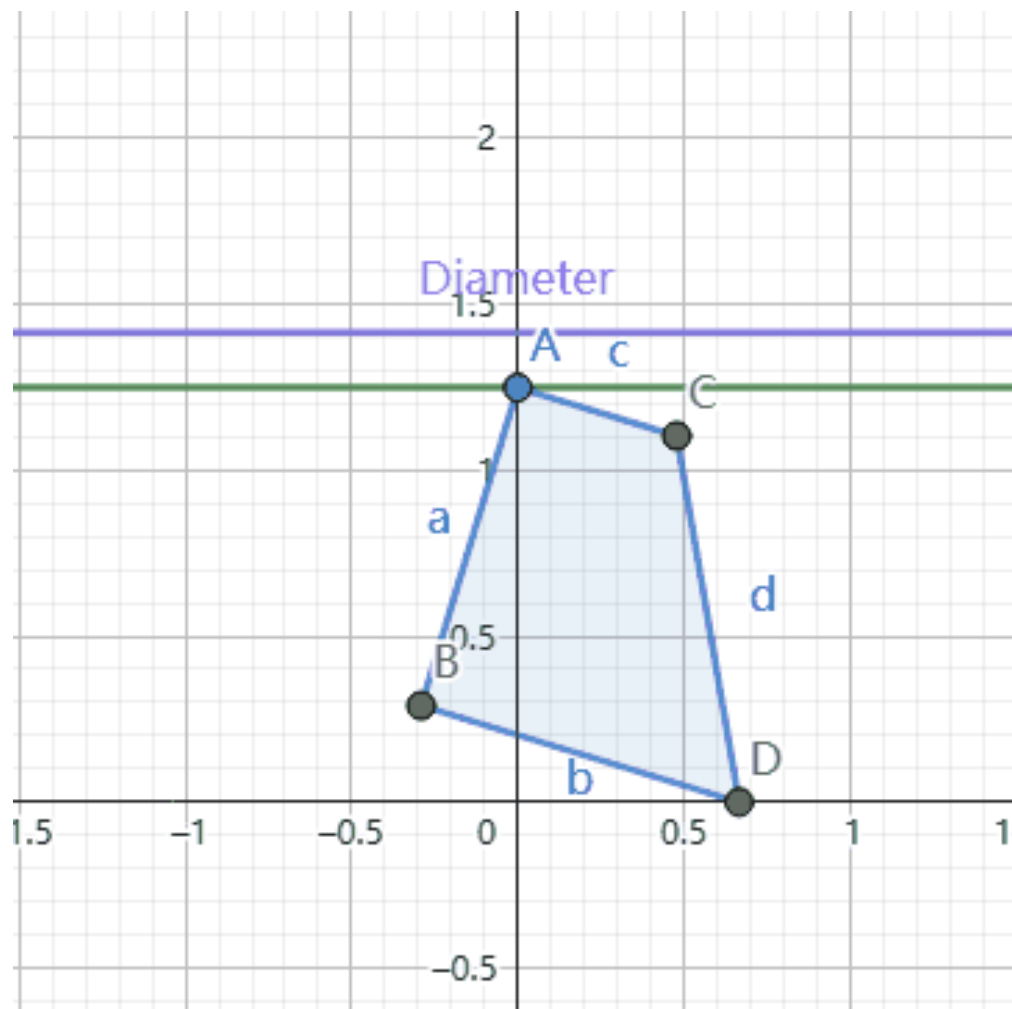
数据范围

- $3 \leq n \leq 50000$



旋转卡壳

考虑“旋转”这个凸包，即可求出直径。（若无法渲染，点击图片打开 GIF）



旋转卡壳

然而在实际实现中，我们只旋转“卡住凸包的直线”，不旋转凸包本身。

大多数的旋转卡壳题可以证明只需要在**某条直线和凸包的边重合**时去相应计算答案。

比如求直径时，只需要在**某条直线和凸包的边重合**时，将边的两个端点与另一条直线的切点分别去更新直径答案即可。

证明：

- 考虑凸包直径某端旋转到“地面”上的情况。（“地面”指前面 GIF 的 x 轴）
- 如果此时没有将直径更新入答案，那么相当于对面直线把直径另一端“跳过”了。
- 那么在另一端旋转到“地面”时，直径的这一段一定不会被“跳过”。



旋转卡壳

其实有拓展性更高的写法，其更擅长处理取到答案时，**不一定有某条直线和凸包的边重合**的情况。

- 对于每次旋转，看两条直线旋转到下一条边所需旋转的角度。
- 选择角度小的一条进行旋转。

这样做的好处：

- 在一些简单的问题中，**答案在重合处计算即可**的结论会更加显然或方便证明。
- 这样每次只有**恰好一条直线跳过了一个点**，旋转过程中的答案可以方便被表示，然后可以当作函数区间最值问题求解。



旋转卡壳

[HNOI2007] 最小矩形覆盖

Time Limit: 1 sec / Memory Limit: 512 MB

给定平面上 n 个点，求能够覆盖所有点的最小面积的矩形。

数据范围

- $3 \leq n \leq 50000$



旋转卡壳

这里卡凸包的直线变成了四条，道——↗↘ 理~←→↑↓ 是一样的。

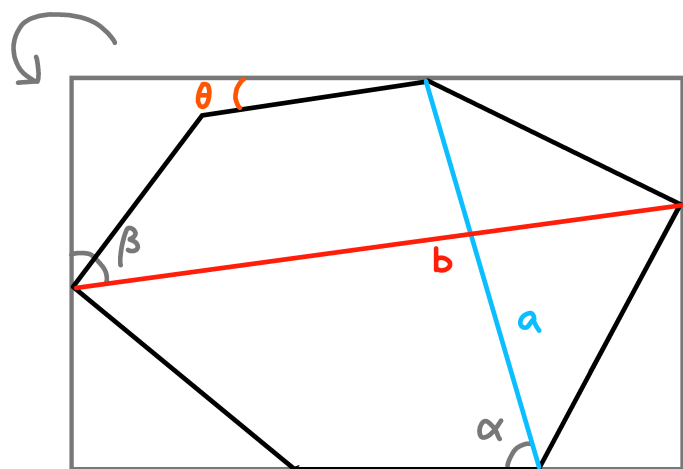
你可以猜答案在**某直线与凸包的边重合**时取到，然后就直接过了。

但是不提倡这样的做法，这里我们先考虑拓展性更高的旋转卡壳方法。



旋转卡壳

考虑某次旋转过程。



假设这次旋转到底经过角度为 $\theta_{\max}$ ，当前旋转角度为 θ 。
此时面积可以表示为 $ab \cdot \sin(\alpha + \theta) \cdot \sin(\beta + \theta)$ 。

旋转卡壳

$$ab \cdot \sin(\alpha + \theta) \cdot \sin(\beta + \theta) = ab \cdot \frac{1}{2}(\cos(\alpha - \beta) - \cos(\alpha + \beta + 2\theta))$$

为了让面积尽可能小，我们要让 $\cos(\alpha + \beta + 2\theta)$ 尽可能大。

这里其实已经可以转化为 $\cos$ 函数区间求最值问题了，但是我们还可以挖掘下性质。

注意到 $0 \leq \alpha \leq \alpha + \theta \leq \pi$, $0 \leq \beta \leq \beta + \theta \leq \pi$ 。

那么就有 $0 \leq \alpha + \beta \leq \alpha + \beta + 2\theta \leq 2\pi$ 。

故这个 $\cos$ 上的区间求最值是在 $[0, 2\pi]$ 之间的，显然最大值在某端点取到。

也就是意味着答案一定在**某直线与凸包的边重合**时取到，直接做就可以了。



半平面交

半平面：一条直线将平面划分为两部分，其中一侧的点集构成一个**半平面**。

根据是否将直线上的点包含进去，可以分为开半平面和闭半平面。

解析式一般为 $Ax + By + C \geq 0$ 或 $Ax + By + C > 0$ 。

这里我们用一个点 P 和一条向量 $\vec{v}$ 表示一个半平面，
其表示点集 $\{X | \vec{v} \times (X - P) \geq 0, X \in \mathbb{R}^2\}$ 。

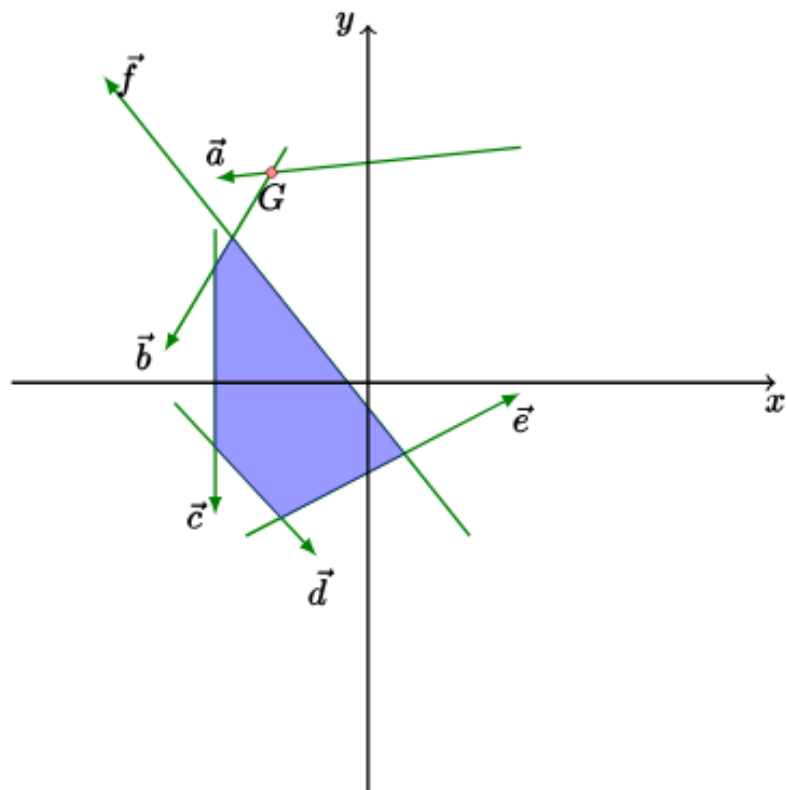
不包含直线的话只需在实际使用的时候判掉边界即可。



半平面交

半平面交是若干半平面的交集，其形状可能有三种情况：

- 凸多边形（封闭）
- 凸壳（不封闭）
- 空集（无交）



半平面交

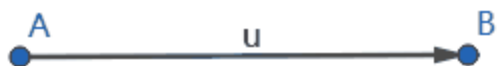
我们先考虑交为凸多边形的时候，可以使用如下流程求半平面交：

- 避免 Corner Case，所有极角相同的元素只保留最靠内侧的。
- 将所有点和向量的二元组按照向量的极角大小从小到大排序。
- 按排序后的顺序一个个加入二元组，用一个单调队列维护当前的二元组序列。
- 当前考虑到第 i 个二元组时，先不加入。如果队尾有至少两个元素，且交点在当前半平面外，就不断弹出队尾。
- 如果队首有至少两个元素，且交点在当前半平面外，就不断弹出队首。
- 插入第 i 个二元组。
- 在考虑完所有二元组后，用队首元素将队尾多余元素弹出。



半平面交

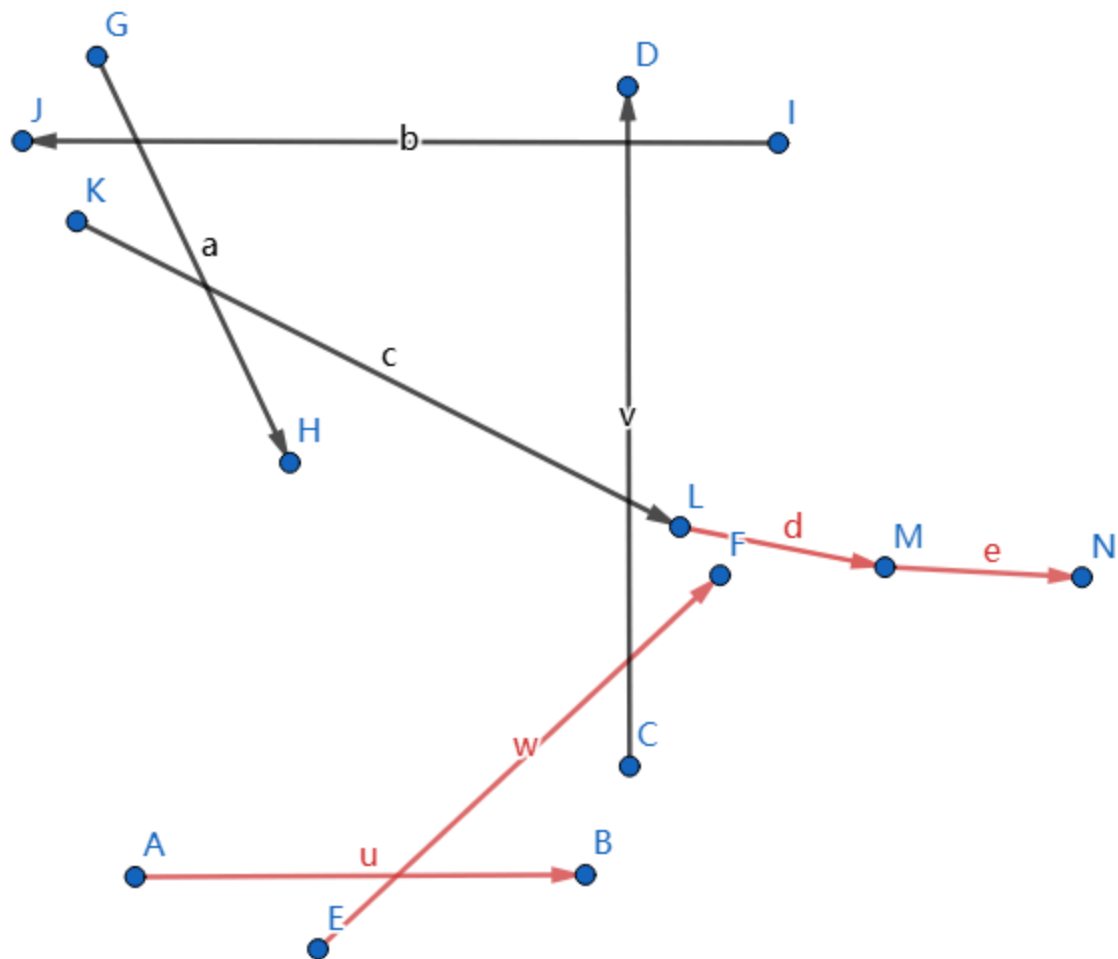
(若无法渲染, 点击图片打开 GIF)



半平面交

为了理解这个算法，我们考虑按顺序加入，但是只弹出队尾得到的轨迹。

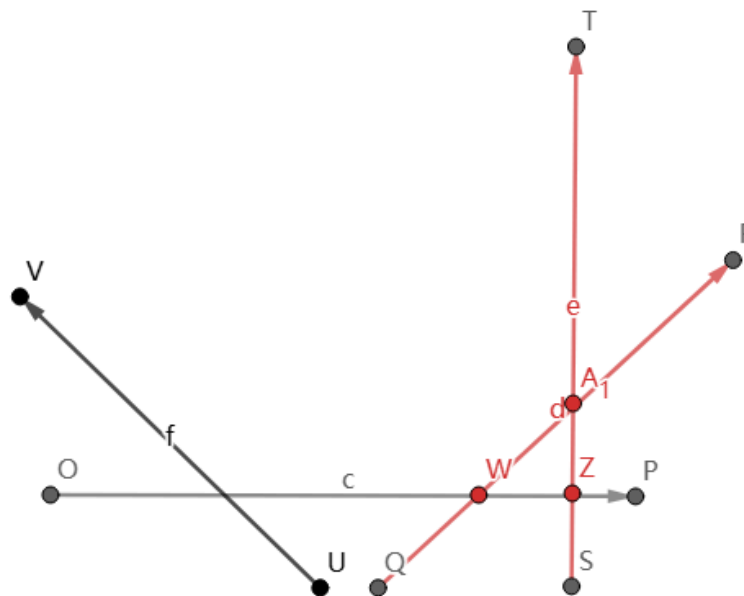
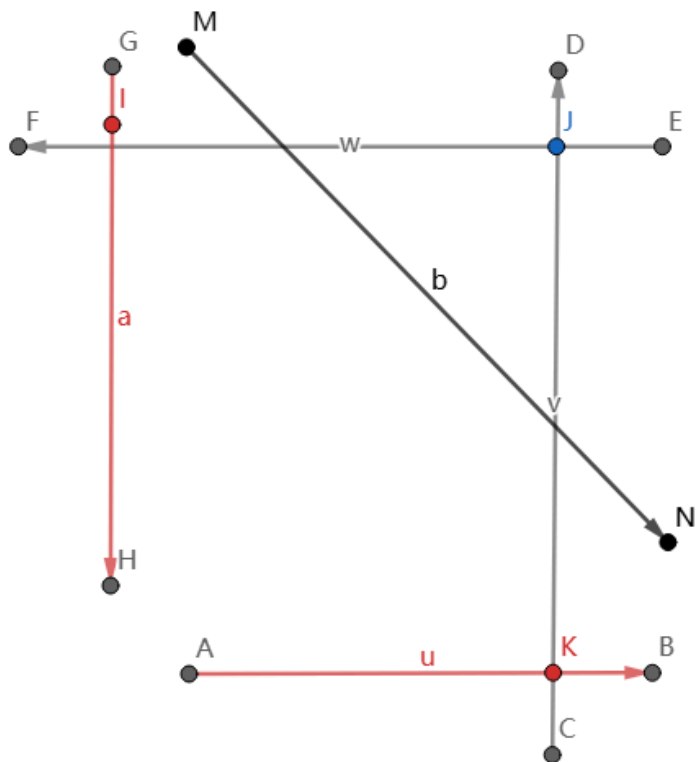
弹出队首和最终弹出队尾多余元素，实际上是在将围成凸多边形前后的多余元素弹出。



半平面交

关于为什么要先弹队尾再弹队首：

- 左图是第一种需要同时弹出两端的情况，此时队首队尾两者弹出先后顺序无影响。
- 右图是第二种需要同时弹出两端的情况，此时所有交点都在外侧，应只保留队首元素，故先弹队尾可以保证正确性。

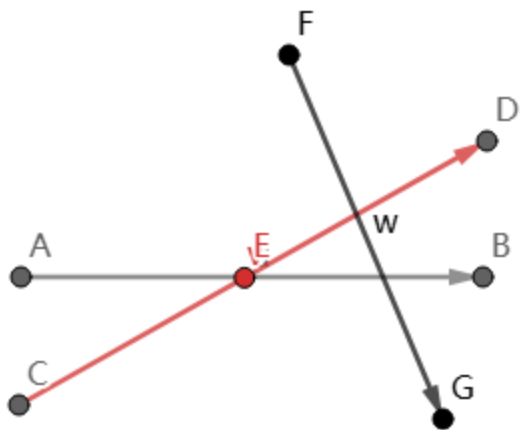


UNOS.
2115

半平面交

现在考虑交集不封闭的情况。

我们发现直接做会出现这种问题：



在插入黑色二元组 $\vec{w}$ 时，会将队尾弹出，但是此时我们显然应该弹出另一条。

UNOS.

半平面交

原因是当交集不封闭时，存在相邻两个二元组的向量极角之差 $> 180^\circ$ 。

显然这种情况只会出现至多一对相邻元素中。

所以我们有三种处理方法：

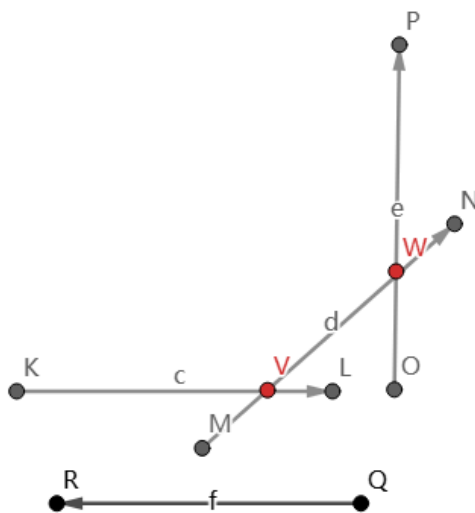
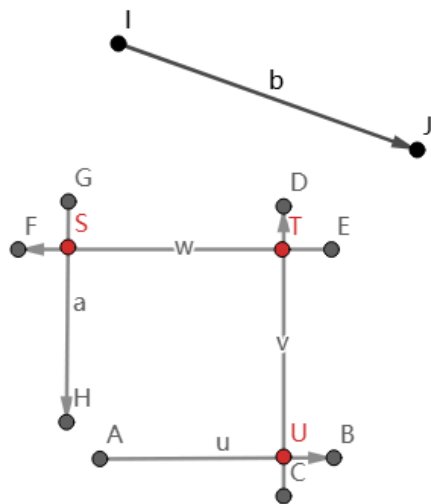
- 整体循环位移，将这对相邻元素的后者作为极角最小的元素。
- 在平面的最外面加入一个长方形的边界，注意要足够大且保证精度。
- 在出现这种情况时特判如何弹出。（不推荐）



半平面交

最后是空集的情况。考虑空集一定是在加入某元素后形成的，有两种情况：

- 之前形成一个封闭凸多边形，此时将弹到只剩队首和当前元素，并且两元素的向量极角之差 $> 180^\circ$ 。
- 之前形成一个不封闭的凸壳，此时将弹到只剩队首和当前元素，并且两元素的向量极角之差 $\geq 180^\circ$ 。



半平面交

在这之后，继续插入的元素与队首元素极角之差都 $> 180^\circ$ ，

它们不会再弹出队首，而在最后，弹掉队尾多余元素会让最后队内只剩下 2 个元素。

所以在保证**围成的区域是封闭的或者为空**的情况下，
只需判断最后是否剩下至少 3 个元素。

否则你需要判断队首队尾两个元素的位置关系：

- 极角之差 $> 180^\circ$ 时，交集为空。
- 极角之差 $< 180^\circ$ 时，交集为不封闭的凸壳。
- 极角之差 $= 180^\circ$ 时，具体根据它们的相对位置确定交集大小。



半平面交

[CF696F] ...Dary!

Time Limit: 3 sec / Memory Limit: 256 MB

给出一个平面上的 n 个点的凸多边形，保证没有三点共线。

若将所有边的两端无限延长成直线，要求求出一个最小的非负实数 r ，

使得可以在平面上放两个半径为 r 的圆，满足所有直线都和至少一个圆有交。

数据范围

$$3 \leq n \leq 300$$



半平面交

考虑二分答案。

这样每条直线可以往两侧移动 r ，中间夹着的区域表示需要有至少一个圆心落在内部。

可以证明一定是把直线按照多边形上的顺序，分成两个区间，这两个区间的直线分别用一个圆心去满足条件。

那么考虑双指针，从每条直线开始，求出从它开始的一个极大的区域有交区间，是否有交跑一遍半平面交即可。

单次判断合法是 $O(n^2 \log n)$ 的，可以在最初进行极角排序优化至 $O(n^2)$ 。

套上外层二分，时间复杂度 $O(n^2(\log |V| + |\log \epsilon|))$ 。



GL & HF